COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	August 1, 2026	**DATE SUBMITTED:**	August 4, 2026
NAME:	Mendoza, Nathaniel B,	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 Stacks

1. PROCEDURE OUTPUT

ILO A

Screenshot

C: > Users > mendo > OneDrive > Documents > C++ > Stacks > ILOA.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main () {
    std::stack<int>stack1;
    std::stack<int>stack2;

    std::cout << "Testing the Stack STL" << std::endl;
    std::cout<< "Is the stack empty? " << stack1.empty();

    /* PUSH */
    std::cout << "\nTesting STACK PUSHING" << std::endl;
    stack1.push(10);
    std::cout << "The top of the stack is: " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout << "The top of the stack is: " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout << "The top of the stack is: " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout << "The top of the stack is: " << stack1.top() << std::endl;

    /* POP */
    std::cout << "\nTesting STACK POPPING" << std::endl;
    stack1.pop();
    std::cout << "The top of the stack is: " << stack1.top() << std::endl;

    std::cout <<  "Is the stack empty? " << stack1.empty() << std::endl;

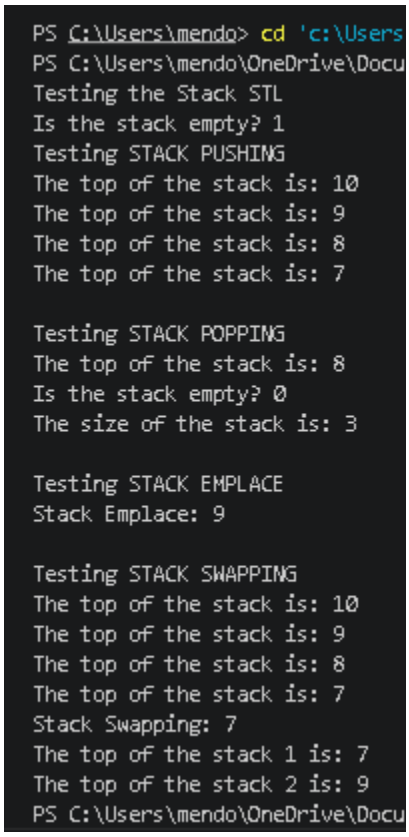
    std::cout << "The size of the stack is: " << stack1.size() << std::endl;

    /* STACK EMPLACE*/
    std::cout << "\nTesting STACK EMPLACE" << std::endl;
    std::cout << "Stack Emplace: " << stack1.emplace(9) << std::endl;

    /* STACK SWAP */
    std::cout << "\nTesting STACK SWAPPING" << std::endl;
    stack2.push(10);
    std::cout << "The top of the stack is: " << stack2.top() << std::endl;
    stack2.push(9);
    std::cout << "The top of the stack is: " << stack2.top() << std::endl;
    stack2.push(8);
    std::cout << "The top of the stack is: " << stack2.top() << std::endl;
    stack2.push(7);
    std::cout << "The top of the stack is: " << stack2.top() << std::endl;

    stack1.swap(stack2);

    std::cout << "Stack Swapping: " << stack1.top() << std::endl;
    std::cout << "The top of the stack 1 is: " << stack1.top() << std::endl;
    std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;
}
```

Output	
Analysis	Using the C++ STL made it easier to understand the basic stack operations because most of the work was already handled by the built-in library. By observing the push, pop, and top operations, I was able to clearly see how the LIFO (Last In, First Out) principle works in practice.

COMPUTER ENGINEERING

QUEZON CITY

ILO B1

Screenshot

ILOB1.cpp

C: > Users > mendo > OneDrive > Documents > C++ > Stacks > ILOB1.cpp > display()

```cpp
#include <iostream>

/* GLOBAL DECLARATION */
#define maxCap 10

const size_t arrSize = 10;
int stackArr[maxCap];
int top = 1, newData;

/* PROTOTYPE FUNCTIONS */
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main() {

    int choice;
    /* MAIN DRIVER VROOM VROOM */
    while(true){
        std::cout << "================================================================" << std::endl;
        std::cout << "Stack Operations \n" <<  std::endl;
        std::cout << "1.PUSH\n2.POP\n3.TOP\n4.isEmpty\n5.isFull\n6.Display" << std::endl;
        std::cout << "================================================================" << std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push(); break;
            case 2: pop(); break;
            case 3: Top(); break;
            case 4: std::cout << "Is stack empty? " << isEmpty() << std::endl; break;
            case 5: std::cout << "Is stack full? " << isFull() << std::endl; break;
            case 6: display(); break;
            default: std::cout << "Unalived Choice." << std::endl;
            break;
        }
    }
}

/* FUNCTION DEFINITION */

bool isEmpty() {
    /* HOW TO VERIFY IF STACK IS EMTPY */
    if(top == -1) return true;
    return false;
}

bool isFull(){
    if(top == maxCap - 1) return true;
    return false;
}

void push(){
```

```
    // ERROR CHECKING
    if(isFull()){
        std::cout << "Stack Overflow " << std::endl;
        return;
    }
    // PUSHING STACK
    std::cout << "\nEnter a new Value: " << std::endl;
    std::cin >> newData;
    // HOW TO INSERT THE DATA IN THE STACK
    stackArr[++top] = newData;
}

void pop(){
    // ERROR CHECKING
    if(isEmpty){
        std::cout << "Stack Underflow" << std::endl;
    }

    // DISPLAY THE VALUE THAT WE ARE GOING TO POP IN POP IN BLOOM
    std::cout << "POPPING: " << stackArr[top] << std::endl;

    // DECREMENT THE TOP VALUE FRPM THE STACK
    top--;
}

void Top(){
    // ERROR CATCHING
    if(isEmpty){
        std::cout << "The Stack is empty" << std::endl;
        return;
    }
    // CHECK TOP VALUE
    std::cout << "Top Element: " << stackArr[top] << std::endl;
}

void display() {
    if (isEmpty()) {
        std::cout << "Stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (TOP TO BOT):" << std::endl;
    for (int j = top; j >= 0; j--) {
        std::cout << stackArr[j] << " ";
    }
    std::cout << std::endl;
}
```

Output

```
PS C:\Users\mendo> cd 'c:\Users\mendo\OneDrive\Documents\C++\Stacks\output
PS C:\Users\mendo\OneDrive\Documents\C++\Stacks\output> & .\'ILOB1.exe'
====================================================================
Stack Operations

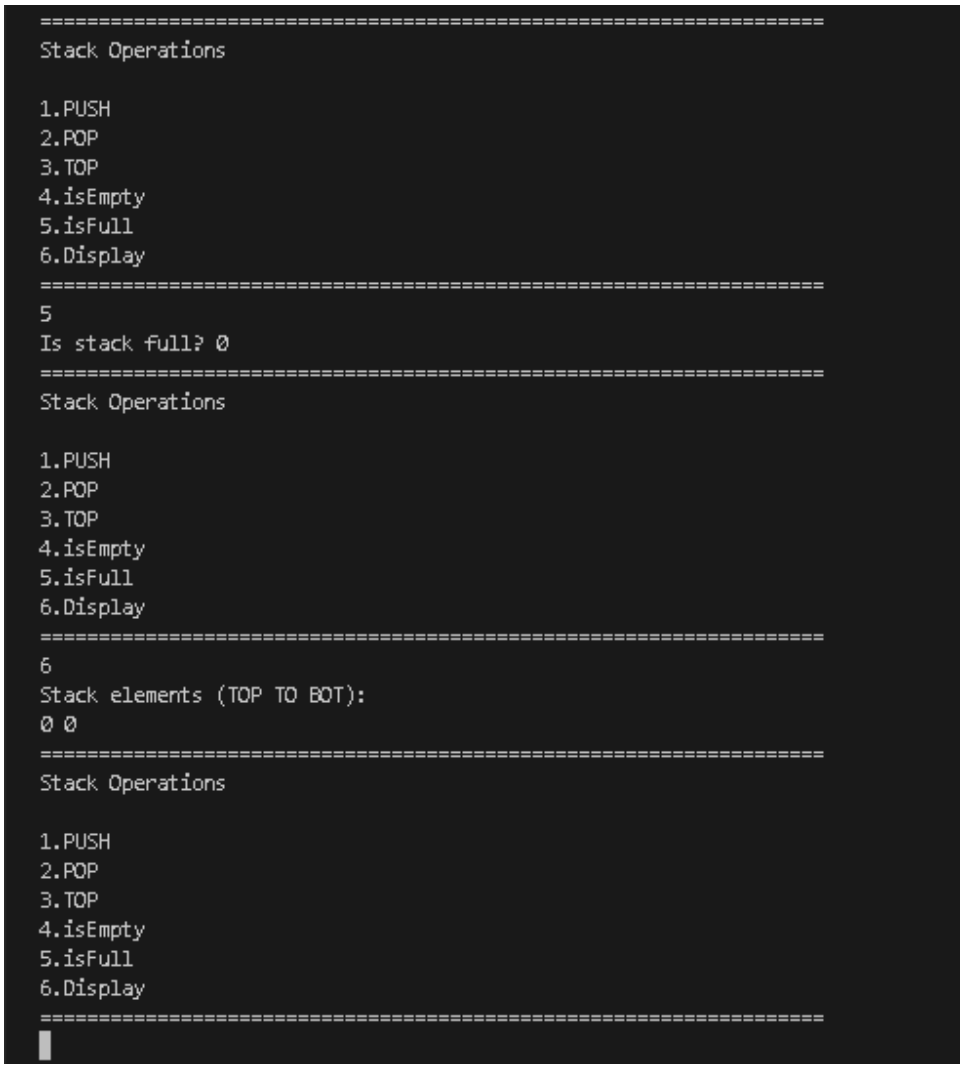
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.Display
====================================================================
1

Enter a new Value:
1
====================================================================
Stack Operations

1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.Display
====================================================================
2
Stack Underflow
POPPING: 1
====================================================================
Stack Operations

1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.Display
====================================================================
3
The Stack is empty
====================================================================
Stack Operations

1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.Display
====================================================================
4
Is stack empty? 0
====================================================================
```



Analysis	Implementing the stack with an array helped me understand how the top index controls the movement of elements during push and pop operations. I also noticed that arrays have a fixed size, so the stack cannot grow beyond its assigned capacity, which can lead to overflow.

ILO B2

main.cpp

Screenshot

```cpp
#include <iostream>
#include "stackList.h"

int main() {

    std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---\n";

    //PUSHING
    push('C');
    push('P');
    push('E');

    //DISPLAY THE CURRENT TOP

    Top<char>; /* TOP ELEMENT SHOULD BE 'E' */
    displayALL<char>();
    std::cout << "\n --- POPPIING ALL ELEMENT ---\n";

    while(!isEmpty<char>()) {
        std::cout<<"Popped: "<< pop<char>() <<std::endl;
    }

    std::cout << "\n --- TESTING THE UNDERFLOW CONDITION --- \n";
    pop<char>(); /* UNDERFLOW CONDITION */

    return 0;
}
```

stackList.h

Screenshot

ILOB2main.cpp | stacklist.h

C: > Users > mendo > OneDrive > Documents > C++ > Stacks > ILOB2 > stacklist.h > ...

```cpp
#ifndef STACKLIST_H
#define STACKLIST_H

#include <iostream>

template <typename T>
class Node {
    public:
    T data; // VALUE WE NEED TO STORE
    Node* next; // POINTER TO THE NEXT NODE
};

// GLOBAL POINTER FOR THE TOP OF THE STACK
template <typename T>
Node<T>* head = nullptr;

// OPERATTIONS FOR THE STACK

// PUSH
template <typename T>
void push(T newData) {
    // CREATE A NEW NODE AND ASSIGN THE newData
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;

    // NEW NODE POINTS TO CURRENT TOP, THEN BECOMES THE NEW TOP
    newNode->next = head<T>;
    head<T> = newNode;
}

// POP

template <typename T>
T pop() {
    // CHECK IF THE STACK IS EMPTY:
    if (head<T> == nullptr) {
        std::cerr << "Stack Underflow: Cannot pop from an empty stack." << std::endl;
    }

    // SAVE THE CURRENT TOP NODE
    Node<T>* temp = head<T>;
    // STORE ITS DATA
    T tempVal = temp->data;

    // MOVE THE TOP POINTER TO THE NEXT NODE
    head<T> = head<T>->next;

    // FREE THE REMOVED NODE
    delete temp;

    // RETURN THE REMOVED VALUE
    return tempVal;
}

// isEmpty
```

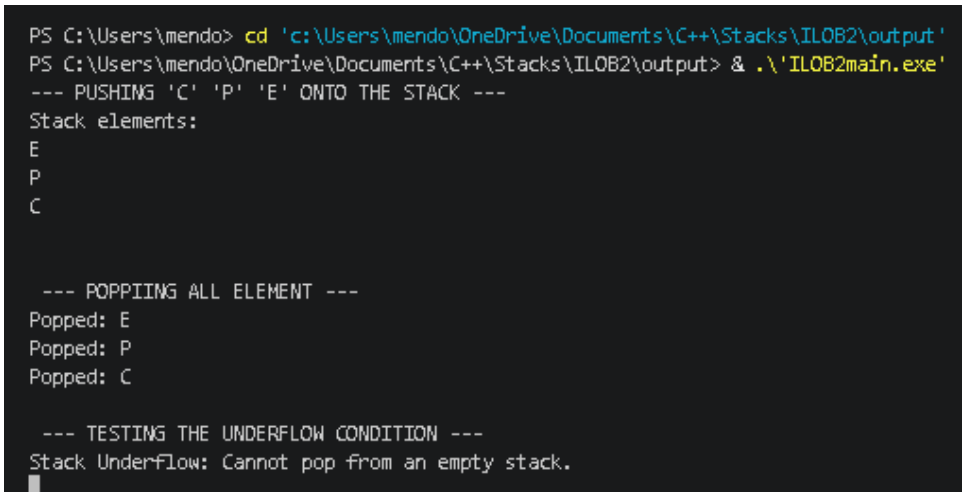
```cpp
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

// Top
template <typename T>
void Top() {
    // ERROR CATCHING:
    if (head<T> == nullptr) {
        std::cout << "Stack is empty. No top element." << std::endl;
        return;
    }
    // DISPLAY THE TOP ELEMENT
    std::cout << "Top element: " << head<T>->data << std::endl;
}

// DISPLAY ALL
template <typename T>
void displayALL() {
    // ERROR CATCHING:
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow\n" << std::endl;
        return;
    }

    // TRAVERSE THE STACK AND DISPLAY ALL ELEMENTS
    Node<T>* temp = head<T>;
    std::cout << "Stack elements: \n";
    while (temp != nullptr) {
        std::cout << temp->data << " \n";
        temp = temp->next;
    }
    std::cout << std::endl;
}

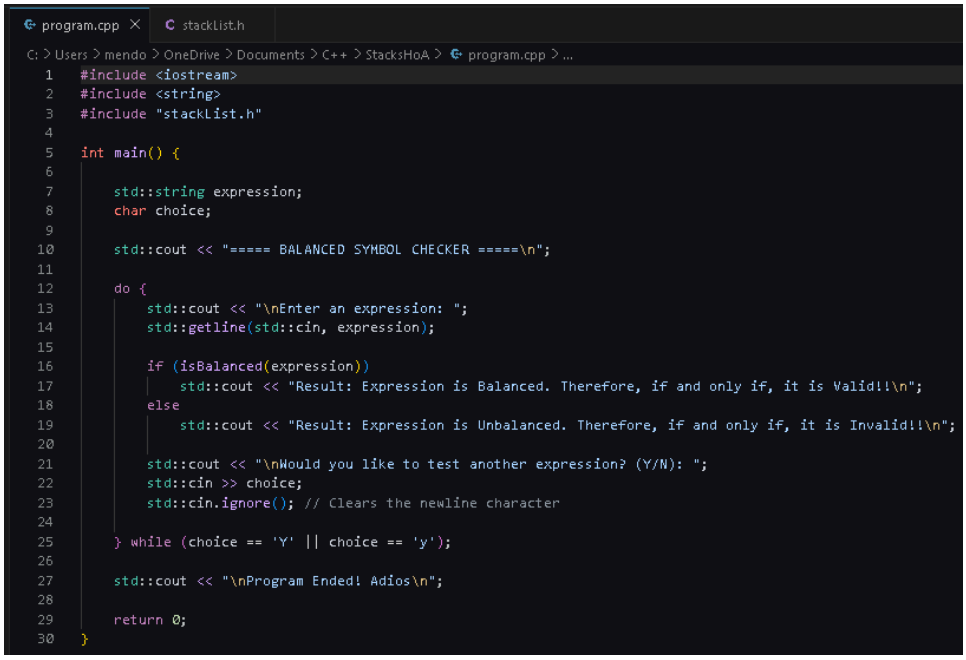
#endif // STACKLIST_H
```

Output	
Analysis	Creating a stack using a linked list showed me that the stack can grow and shrink dynamically without worrying about a fixed size. I found it easier to visualize that each new node becomes the new top of the stack, while the previous top simply moves to the next position, making push and pop operations more flexible.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

program.cpp
Screenshot



```cpp
#include <iostream>
#include <string>
#include "stackList.h"

int main() {

    std::string expression;
    char choice;

    std::cout << "===== BALANCED SYMBOL CHECKER =====\n";

    do {
        std::cout << "\nEnter an expression: ";
        std::getline(std::cin, expression);

        if (isBalanced(expression))
            std::cout << "Result: Expression is Balanced. Therefore, if and only if, it is Valid!!\n";
        else
            std::cout << "Result: Expression is Unbalanced. Therefore, if and only if, it is Invalid!!\n";

        std::cout << "\nWould you like to test another expression? (Y/N): ";
        std::cin >> choice;
        std::cin.ignore(); // Clears the newline character

    } while (choice == 'Y' || choice == 'y');

    std::cout << "\nProgram Ended! Adios\n";

    return 0;
}
```

stackList.h	
Screenshot	

```cpp

    if (head<T> == nullptr) {
        std::cout << "Stack is empty. No top element." << std::endl;
        return;
    }

    std::cout << "Top element: " << head<T>->data << std::endl;
}

// DISPLAY ALL
template <typename T>
void displayALL() {

    if (head<T> == nullptr) {
        std::cout << "Stack Underflow\n";
        return;
    }

    Node<T>* temp = head<T>;

    std::cout << "Stack Elements:\n";

    while (temp != nullptr) {
        std::cout << temp->data << std::endl;
        temp = temp->next;
    }

    std::cout << std::endl;
}

// ========================================
// SUPPLEMENTARY ACTIVITY FUNCTIONS
// ========================================

// CHECK IF OPENING SYMBOL
inline bool isOpening(char ch) {
    return ch == '(' || ch == '[' || ch == '{';
}

// CHECK IF CLOSING SYMBOL
inline bool isClosing(char ch) {
    return ch == ')' || ch == ']' || ch == '}';
}

// CHECK IF SYMBOLS MATCH
inline bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

// CHECK IF EXPRESSION IS BALANCED
inline bool isBalanced(const std::string& expression) {
```

```
109
110     // Clear stack before checking
111     while (!isEmpty<char>()) {
112         pop<char>();
113     }
114
115     for (char ch : expression) {
116
117         if (isOpening(ch)) {
118             push<char>(ch);
119         }
120         else if (isClosing(ch)) {
121
122             if (isEmpty<char>())
123                 return false;
124
125             char topSymbol = pop<char>();
126
127             if (!isMatching(topSymbol, ch))
128                 return false;
129         }
130     }
131
132     return isEmpty<char>();
133 }
134
135 #endif // STACKLIST_H
```

Output

```
Enter an expression: (A+B)+(C-D)
Result: Expression is Balanced. Therefore, if and only if, it is Valid!!

Would you like to test another expression? (Y/N): y

Enter an expression: ((A+B)+(C-D)
Result: Expression is Unbalanced. Therefore, if and only if, it is Invalid!!

Would you like to test another expression? (Y/N): y

Enter an expression: ((A+B)+[C-D])
Result: Expression is Balanced. Therefore, if and only if, it is Valid!!

Would you like to test another expression? (Y/N): y

Enter an expression: ((A+B]+[C-D]}
Result: Expression is Unbalanced. Therefore, if and only if, it is Invalid!!

Would you like to test another expression? (Y/N): n

Program Ended! Adios
```

Analysis

Implementing the balanced symbol checker showed me how a stack can be applied to

	solve a practical problem instead of only performing basic push and pop operations. Testing different expressions helped me understand that the validity of an expression depends on every opening symbol having the correct matching closing symbol in the proper order.

Expression	Valid? (Y/N)	Output(Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter an expression: (A+B)+(C-D) Result: Expression is Balanced. Therefore, if and only if, it is Valid!!	Has all opening and closing parenthesis so valid.
((A+B)+(C-D)	N	Enter an expression: ((A+B)+(C-D) Result: Expression is Unbalanced. Therefore, if and only if, it is Invalid!!	Expressions have 1 extra opening parenthesis so invalid.
((A+B)+[C-D])	Y	Enter an expression: ((A+B)+[C-D]) Result: Expression is Balanced. Therefore, if and only if, it is Valid!!	Has equal parenthesis and brackets opening and closing so valid.
((A+B]+[C-D]}	N	Enter an expression: ((A+B]+[C-D]} Result: Expression is Unbalanced. Therefore, if and only if, it is Invalid!!	Expressions has missing bracket so invalid.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.